CHAPTER 08

Vagrant

Infrastructure as Code with VirtualBox

From a Golden Ubuntu Image to a fully automated, twelve-server DevSecOps laboratory — defined entirely in code.

Packer | **Vagrant** | VirtualBox | Ansible

CHAPTER GOAL

From One Golden Image to Twelve Live Servers

In Chapter 07 we built a Golden Ubuntu Image with Packer.

- Ubuntu Server
- Security hardening
- Base packages
- SSH
- Docker prerequisites
- Monitoring tools
- Cleanup

THE PROBLEM

Manually creating twelve VirtualBox VMs by hand — every time.

THE ANSWER

Vagrant — deploy the entire infrastructure automatically, as Infrastructure as Code.

By the end of this chapter you will be able to:

- ✓ Install Vagrant & import the Golden Image
- ✓ Configure networking, CPU, RAM & hostnames
- ✓ Generate an Ansible inventory automatically
- ✓ Provision all lab VMs from one Vagrantfile
- ✓ Manage the full VM lifecycle
- ✓ Version-control the entire lab

LEARNING OBJECTIVES

What You Will Understand After This Chapter

1

Infrastructure as Code

Describing infra in versioned files, not clicks

2

Vagrant Architecture

How boxes, providers, and the Vagrantfile fit together

3

Vagrant Boxes

Reusable OS templates built from Packer images

4

Providers

VirtualBox as the execution engine behind Vagrant

5

Multi-Machine Environments

Defining many VMs from one configuration

6

Reproducible Infrastructure

Rebuilding the same lab anywhere, on demand

7

VirtualBox Automation

CPU, RAM, disks & networking — all in code

The Orchestrator of Your Local Infrastructure

VirtualBox

Creates virtual machines

Packer

Builds virtual machine images

Ansible

Configures operating systems

Vagrant

Connects all of these together

Think of Vagrant as the orchestrator of your local infrastructure.

It doesn't build images and it doesn't configure operating systems — it takes the pieces built by Packer and VirtualBox, and the configuration handled later by Ansible, and turns them into a single, reproducible, push-button deployment.

WORKFLOW COMPARISON

Traditional Clicking vs. Vagrant Automation

TRADITIONAL WORKFLOW

1. Open VirtualBox
2. Create VM
3. Allocate RAM
4. Allocate CPU
5. Attach disk
6. Configure network
7. Boot
8. *Repeat 12 times*

VAGRANT WORKFLOW

```
Write the Vagrantfile
```

▼

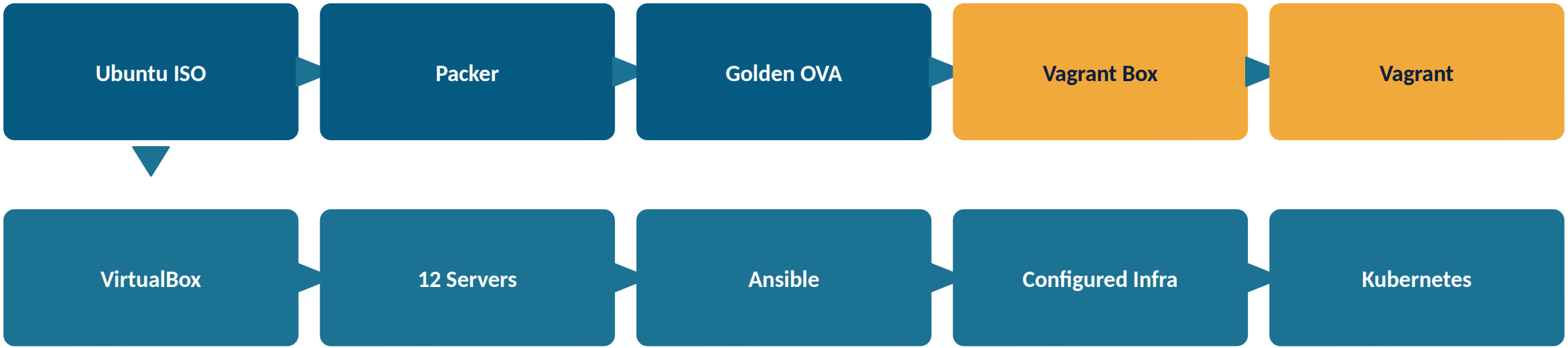
```
Run vagrant up
```

▼

```
12 virtual machines — done
```

Everything is described in code.
No clicking. No forgotten settings. No manual mistakes.

The Full Provisioning Pipeline



This is the workflow used throughout the remainder of the book.

Vagrant sits at the hinge point: it consumes the image Packer built and hands a running, configured fleet of servers to Ansible.

VirtualBox Is a Hypervisor — Not Infrastructure as Code

VirtualBox is an excellent hypervisor. **However, VirtualBox itself is not Infrastructure as Code.**

Imagine rebuilding your laptop. Would you like to remember, for every single VM:

- RAM for every VM
- CPU count
- IP addresses
- Hostnames
- Network adapters
- Boot order
- Storage controllers

Probably not.

IaC

Vagrant stores all of those settings inside a single file.

One Vagrantfile becomes the single source of truth for the entire lab's hardware and network configuration.

Install Vagrant on Ubuntu

```
sudo apt update
sudo apt install wget gnupg software-properties-common

wget -O- https://apt.releases.hashicorp.com/gpg \
  | sudo gpg --dearmor \
  -o /usr/share/keyrings/hashicorp-archive-keyring.gpg

echo "deb [signed-by=/usr/share/keyrings/hashicorp-archive-
keyring.gpg] \
  https://apt.releases.hashicorp.com \
  $(lsb_release -cs) main" \
  | sudo tee /etc/apt/sources.list.d/hashicorp.list

sudo apt update
sudo apt install vagrant
```

VERIFY

```
vagrant version
```

Example output:

```
Vagrant 2.4.x
```

HashiCorp maintains an official apt repository, so Vagrant stays updatable through the normal package manager — no manual downloads.

One Repository, Four Responsibilities

```
lab/
├── packer/
│   ├── ubuntu-22.04.pkr.hcl
│   ├── output/
│   └── ubuntu-22.04-base.ova
├── vagrant/
│   ├── Vagrantfile
│   ├── inventory.ini
│   ├── scripts/
│   └── boxes/
├── ansible/
└── kubernetes/
```

Tool	*Responsibility*
`Packer`	Build images
`Vagrant`	Provision VMs
`Ansible`	Configure VMs
`Kubernetes`	Run workloads

Each tool has a single responsibility — Vagrant never builds images and never configures software.

Turning the Golden OVA into a Reusable Box

A Box is a reusable operating system image — think of it as a VirtualBox template.



Every VM in our lab originates from the same box — one build, twelve identical starting points.

CREATE THE BOX FROM CHAPTER 07'S OVA

```
vagrant box add \
  ubuntu22-golden \
  output/ubuntu-22.04-base.ova
```

VERIFY

```
vagrant box list
```

```
ubuntu22-golden  (virtualbox)
```

The Heart of Every Vagrant Project

```
mkdir ~/lab/vagrant
cd ~/lab/vagrant

vagrant init
```

→ *creates Vagrantfile*

```
Vagrant.configure("2") do |config|

end
```

Everything inside this block becomes part of our infrastructure definition.

THE VAGRANTFILE IS EQUIVALENT TO:

- Docker Compose — for containers
- Kubernetes YAML — manifests
- Ansible — Playbooks
- Packer — Templates

ADVANTAGES

- ✓ Repeatable deployments
- ✓ Version control
- ✓ Easy modifications
- ✓ Team collaboration
- ✓ Infrastructure as Code

One SERVERS Hash Describes the Entire Lab

```
config.vm.box = "ubuntu22-golden"

LAB_NETWORK = "DevSecOps-Lab"
DOMAIN      = "devsecops.com"
```

Every VM inherits the exact same operating system. Changing RAM or CPU later means editing only one value.

Hostname	IP	CPU	RAM	Disk
dns-01	10.0.2.10	2	2048	20
idm-01	10.0.2.11	4	6144	40
edge-01	10.0.2.12	4	4096	40
vault-01	10.0.2.13	4	4096	40
gitlab-01	10.0.2.14	4	8192	80
harbor-01	10.0.2.15	4	8192	100
runner-01	10.0.2.16	4	4096	40

Server specifications live in one place — the SERVERS hash — instead of being duplicated twelve times.

One Loop Replaces Twelve Manual Definitions

```
Vagrant.configure("2") do |config|
  config.vm.box = "ubuntu22-golden"
  SERVERS.each do |hostname, settings|
    config.vm.define hostname do |node|
      node.vm.hostname = hostname
    end
  end
end
```

THE LOOP

Executes once for every server in the hash. As the infrastructure grows, no additional code is required — just add a new key.

HOSTNAME EXAMPLES

```
dns-01
idm-01
gitlab-01
vault-01
monitor-01
```

Clear hostnames make SSH sessions and terminal prompts easy to read at a glance.

Every VM Gets Two Network Adapters

ADAPTER 1

NAT

Internet access

ADAPTER 2

Internal Network

Communication between lab machines

```
node.vm.network
"private_network",
  ip: settings[:ip]
```

VirtualBox automatically creates the host-only interface if necessary. Every machine receives a static IP address.

Hostname	IP Address
dns-01	10.0.2.10
idm-01	10.0.2.11
edge-01	10.0.2.12
gitlab-01	10.0.2.14
harbor-01	10.0.2.15

Why two adapters?

NAT gives each VM outbound internet for package installs, while the private network gives every server a predictable, static address that Ansible and your other servers can rely on — no DHCP surprises.

Right-Sizing Every VM Automatically

```
node.vm.provider "virtualbox" do |vb|
  vb.name   = hostname
  vb.memory = settings[:ram]
  vb.cpus   = settings[:cpu]
end
```

GitLab

```
CPU: 4
RAM: 8 GB
```

DNS

```
CPU: 2
RAM: 2 GB
```

Heavier services (GitLab, Harbor) get more CPU and RAM; lightweight services (DNS) stay lean — all from the same SERVERS hash.

```
vb.gui = false

vb.customize ["modifyvm", :id,
  "--nested-hw-virt", "on"]

vb.customize ["modifyvm", :id,
  "--ioapic", "on"]

vb.customize ["modifyvm", :id,
  "--clipboard-mode", "bidirectional"]
```

These settings disable the VirtualBox GUI, enable nested virtualization (needed for Kubernetes/Docker later), enable the I/O APIC, and allow clipboard sharing — improving compatibility and performance.

Under 100 Lines Describes Everything

After combining every configuration section, Vagrant knows:

- ✓ Which box to use
- ✓ VM names
- ✓ CPU count
- ✓ RAM
- ✓ Static IP addresses
- ✓ VirtualBox settings
- ✓ Networking

This is Infrastructure as Code.

VALIDATE BEFORE DEPLOYING

```
vagrant validate
```

✓ Valid

No errors displayed.

✗ Invalid

Vagrant reports the exact line number.

Always validate syntax before running vagrant up on a twelve-machine lab — catching a typo here is far cheaper than debugging a half-booted fleet.

vagrant up — Then vagrant status

```
vagrant up
```

Reading Vagrantfile

Importing box

Creating VM

Configuring CPU/RAM

Configuring network

Booting VM

Waiting for SSH

8 Provision complete

Repeats automatically for every server. A full deployment typically completes in 10–20 minutes.

CHECK STATUS ANY TIME

```
vagrant status
```

```
dns-01            running (virtualbox)
idm-01            running (virtualbox)
edge-01           running (virtualbox)
vault-01          running (virtualbox)
gitlab-01         running (virtualbox)
harbor-01         running (virtualbox)
runner-01         running (virtualbox)
```

vagrant status should become one of your most frequently used commands.

SSH — the Easy Way and the Standard Way

THE VAGRANT WAY

No IPs or keys to remember — Vagrant manages SSH access for you.

```
vagrant ssh dns-01
vagrant ssh gitlab-01
```

Inside the VM:

```
hostname
# dns-01

exit
```

Exit returns you to the host machine.

STANDARD OPENSSH

Ansible and automation scripts often need the standard SSH client instead.

```
vagrant ssh-config dns-01
```

```
Host dns-01
HostName 127.0.0.1
User vagrant
Port 2222
IdentityFile ~/.vagrant.d/insecure_private_key
```

```
ssh -F <(vagrant ssh-config dns-01) default
```

Especially useful when testing Ansible before generating a full inventory.

Control the Whole Lab — or One Machine

WHOLE-LAB COMMANDS

`vagrant up`
Start all VMs

`vagrant halt`
Stop all VMs

`vagrant suspend`
Suspend — saves RAM, resumes fast

`vagrant resume`
Resume suspended VMs

`vagrant reload`
Restart all VMs

`vagrant destroy`
Delete VMs (asks for confirmation)

INDIVIDUAL-MACHINE COMMANDS

`vagrant up dns-01`
Start one VM

`vagrant halt dns-01`
Stop one VM

`vagrant reload gitlab-01`
Restart GitLab

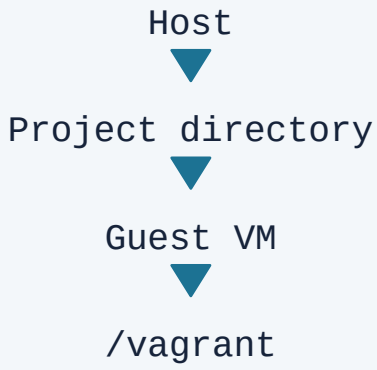
`vagrant destroy harbor-01`
Destroy only Harbor

Because our infrastructure is defined as code, destroying and rebuilding is always just:

```
vagrant destroy
vagrant up
```

Sharing Files In — and Bootstrapping On Boot

SYNCED FOLDERS



Files placed in the project directory become immediately available inside every VM — shell scripts, configs, test data, packages.

AUTOMATIC PROVISIONING

Shell script:

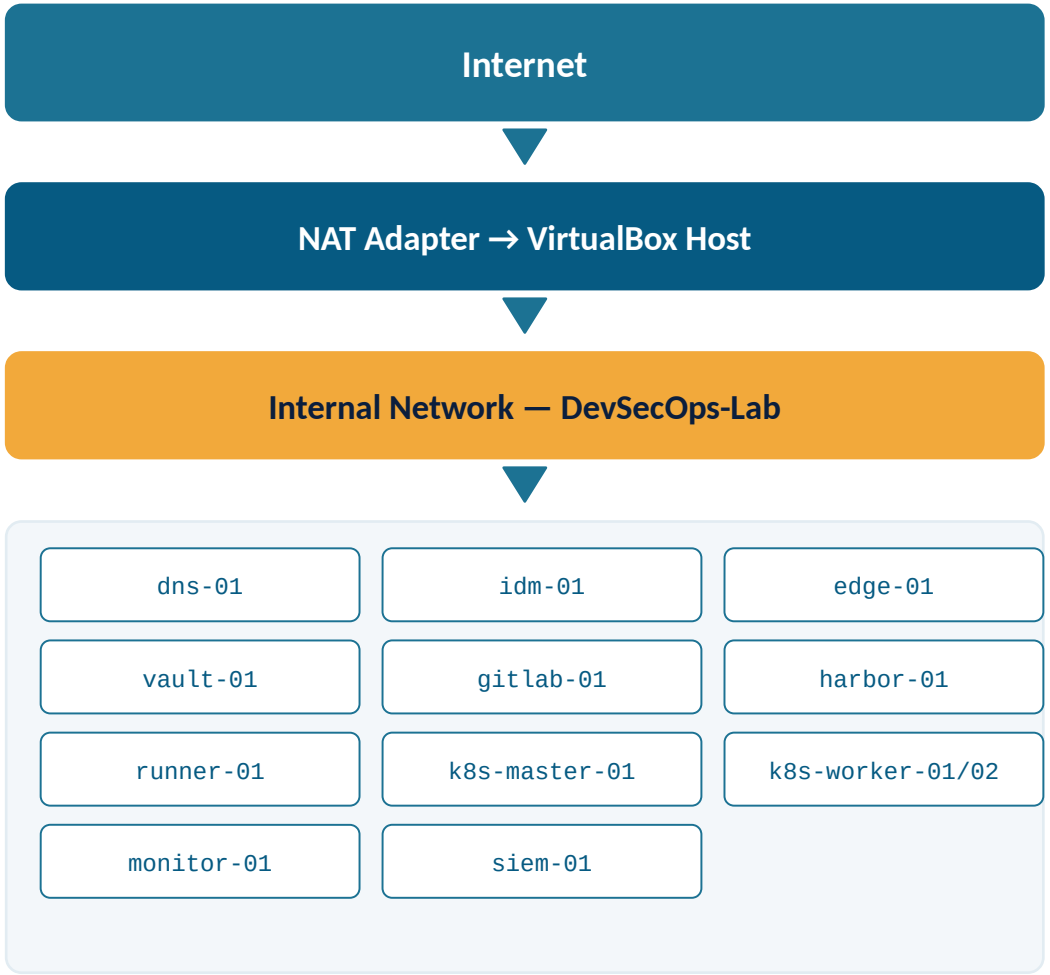
```
node.vm.provision "shell",
  path: "scripts/bootstrap.sh"
```

Ansible playbook:

```
node.vm.provision "ansible" do |ansible|
  ansible.playbook = "../ansible/site.yml"
end
```

For this book, Vagrant handles only infrastructure provisioning — Ansible (next chapter) owns configuration.

Twelve Servers on One Internal Network



Internal network isolation lets servers talk to each other without exposing internal services to the outside world.

Server	vCPU	RAM	Disk
dns-01	2	2 GB	20 GB
idm-01	4	6 GB	40 GB
edge-01	4	4 GB	40 GB
vault-01	4	4 GB	40 GB
gitlab-01	4	8 GB	80 GB
harbor-01	4	8 GB	100 GB
runner-01	4	4 GB	40 GB
k8s-master-01	4	6 GB	60 GB
k8s-worker-01	4	8 GB	80 GB
k8s-worker-02	4	8 GB	80 GB
monitor-01	6	12 GB	120 GB
siem-01	4	8 GB	100 GB

Balanced for a ~64 GB RAM workstation. Reduce allocations during the learning phase if needed.

Safety Nets Before Big Changes

SNAPSHOTS

```
vagrant snapshot save dns-01 clean-install
vagrant snapshot list dns-01
vagrant snapshot restore dns-01 clean-install
vagrant snapshot delete dns-01 clean-install
```

Extremely useful before installing large applications such as GitLab, Harbor, or Kubernetes.

UPDATING THE GOLDEN BOX

Modify Packer template

▼

`packer build`

▼

New OVA

▼

`vagrant box remove ubuntu22-golden`

▼

vagrant box add ubuntu22-golden new-image.ova

▼

`vagrant destroy`

▼

vagrant up

The entire infrastructure is rebuilt from the updated Golden Image — a demonstration of immutable infrastructure.

Your Entire Lab Belongs in Git

```
git init
git add .
git commit -m "Initial Vagrant infrastructure"
```

```
lab/
├── .gitignore
├── README.md
├── LICENSE
├── packer/
├── vagrant/
├── ansible/
├── kubernetes/
└── docs/
```

GIVES YOU

Version control · Team sharing · Backups · Restore on any workstation

NOT EVERYTHING BELONGS IN GIT

.gitignore

```
.vagrant/
*.log
*.tmp
*.box
*.iso
output/
*.retry
*.swp
.DS_Store
```

Generated files should always be recreated, never stored.

Generate the Inventory, Then Validate Connectivity

GENERATE AN ANSIBLE INVENTORY

```
scripts/generate_inventory.sh
```

```
[dns]
dns-01 ansible_host=10.0.2.10

[kubernetes_workers]
k8s-worker-01 ansible_host=10.0.2.21
k8s-worker-02 ansible_host=10.0.2.22

[all:vars]
ansible_user=ubuntu
```

```
chmod +x scripts/generate_inventory.sh
./scripts/generate_inventory.sh
```

VALIDATE CONNECTIVITY

`ping 10.0.2.10`

Ping the DNS server

`ssh ubuntu@10.0.2.10`

Verify SSH

`hostnamectl`

Check hostname

`ip addr`

Check IP address

Repeat these checks for every server before continuing into Ansible.

Your Day-to-Day Command Reference

`vagrant up`	Create and start VMs	`vagrant halt`	Stop VMs
`vagrant reload`	Restart VMs	`vagrant suspend`	Suspend VMs
`vagrant resume`	Resume suspended VMs	`vagrant destroy`	Delete VMs
`vagrant status`	Display VM status	`vagrant ssh <vm>`	Connect to a VM
`vagrant snapshot save`	Create snapshot	`vagrant snapshot restore`	Restore snapshot
`vagrant snapshot delete`	Delete snapshot	`vagrant box list`	List installed boxes

These commands cover nearly all day-to-day management tasks.

Common Issues & Fast Fixes

Provider Not Found

The provider 'virtualbox' could not be found.

```
VBoxManage --version
sudo apt install virtualbox
```

Box Not Found

Vagrant cannot find the box

```
vagrant box list
vagrant box add ubuntu22-golden ...
```

SSH Timeout

vagrant up hangs waiting for SSH

```
sudo systemctl status ssh
```

IP Already In Use

Address already assigned

```
Check SERVERS hash for duplicate IPs
```

VM Already Exists

Machine already exists.

```
Remove or rename the VM in VirtualBox
```

Insufficient Memory

VM fails to boot / host unresponsive

```
vagrant up dns-01 idm-01
```

Always verify VirtualBox itself first — if VirtualBox cannot manage a VM, Vagrant cannot either. (vb list vms)

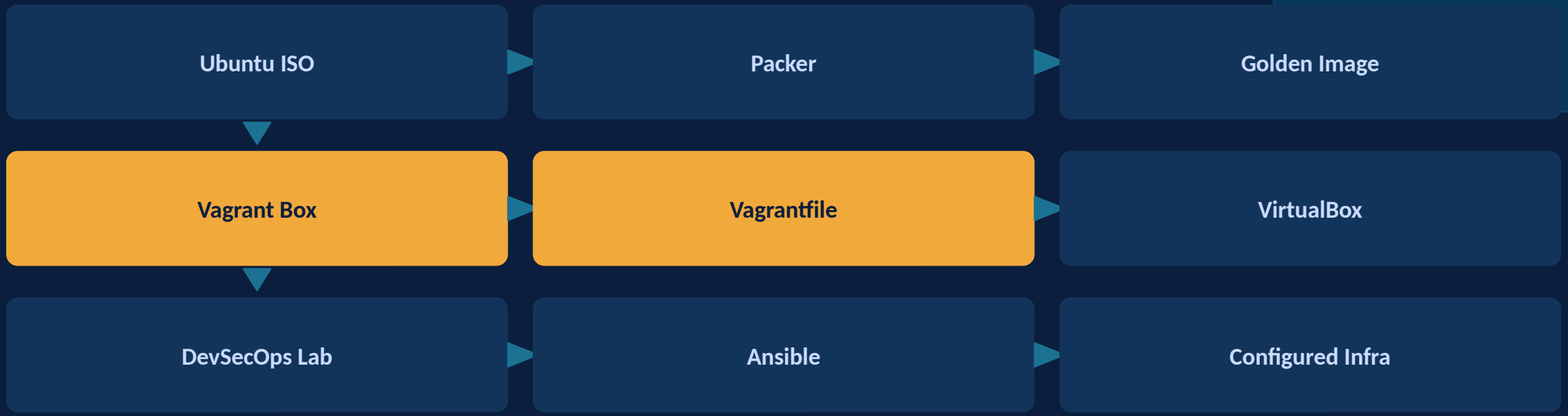
Keep the Lab Predictable and Reproducible

1. Keep the Vagrantfile under version control
2. Store reusable values in variables, not duplicated
3. Never manually modify VM hardware in the VirtualBox GUI
4. Rebuild infrastructure from code whenever changes are needed
5. Update the Golden Image through Packer, not deployed VMs
6. Take snapshots before installing major services
7. Destroy and recreate dev environments instead of repairing them
8. Separate provisioning (Vagrant) from configuration (Ansible)

CHAPTER SUMMARY

Infrastructure as Code, Delivered

The entire DevSecOps laboratory is now defined in a single Vagrantfile — reproducible on any compatible workstation with only a few commands.



WE NOW HAVE

✓ Infrastructure as Code

✓ Reproducible environments

✓ Consistent VM configurations

✓ Automated deployments

✓ Version-controlled infrastructure

✓ Faster disaster recovery

Next: Chapter 09 — Configuring the Lab with Ansible | 28